

Student-Mental-Health-Analysis

August 5, 2025

1 Introduction

In this project, we aim to analyze a dataset which contains responses from students regarding their mental health status during the era of online learning. Our goal is to find out what factors affect students' academic performances with machine learning techniques.

1.1 1. Import Libraries and Data

```
[1]: # Importing the libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib auto
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, \
    classification_report, mean_absolute_error, r2_score, mean_squared_error
```

Using matplotlib backend: module://matplotlib_inline.backend_inline

```
[2]: # Loading the dataset
df = pd.read_csv('Student-Mental-Health-Analysis-During-Online-Learning.csv')
df.head()
```

```
[2]:      Name  Gender  Age  Education Level  Screen Time (hrs/day) \
0   Aarav   Male   15      Class 8      7.1
1   Meera  Female   25         MSc      3.3
2  Ishaan   Male   20        BTech      9.5
3  Aditya   Male   20         BA     10.8
4   Anika  Female   17      Class 11      2.8
```

```
      Sleep Duration (hrs)  Physical Activity (hrs/week)  Stress Level \
```

0	8.9	9.3	Medium
1	5.0	0.2	Medium
2	5.4	6.2	Medium
3	5.6	5.5	High
4	5.4	3.1	Medium

	Anxious Before Exams	Academic Performance Change
0	No	Same
1	No	Same
2	No	Same
3	Yes	Same
4	Yes	Same

1.1.1 1.1 Data Preprocessing

```
[3]: df.shape
```

```
[3]: (1000, 10)
```

```
[4]: # checking for null/missing values
df.isnull().sum()
```

```
[4]: Name          0
Gender          0
Age            0
Education Level  0
Screen Time (hrs/day)  0
Sleep Duration (hrs)  0
Physical Activity (hrs/week)  0
Stress Level    0
Anxious Before Exams  0
Academic Performance Change  0
dtype: int64
```

Wow, data is mad clean.

```
[5]: df.dtypes
```

```
[5]: Name          object
Gender          object
Age            int64
Education Level  object
Screen Time (hrs/day)  float64
Sleep Duration (hrs)  float64
Physical Activity (hrs/week)  float64
Stress Level    object
Anxious Before Exams  object
```

```
Academic Performance Change      object
dtype: object
```

```
[6]: df.duplicated().sum()
```

```
[6]: 0
```

```
[7]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Name                                  1000 non-null   object
1   Gender                                1000 non-null   object
2   Age                                   1000 non-null   int64
3   Education Level                       1000 non-null   object
4   Screen Time (hrs/day)                 1000 non-null   float64
5   Sleep Duration (hrs)                  1000 non-null   float64
6   Physical Activity (hrs/week)          1000 non-null   float64
7   Stress Level                          1000 non-null   object
8   Anxious Before Exams                  1000 non-null   object
9   Academic Performance Change           1000 non-null   object
dtypes: float64(3), int64(1), object(6)
memory usage: 78.3+ KB
```

```
[8]: print(df['Gender'].value_counts())
print(df['Education Level'].value_counts())
print(df['Stress Level'].value_counts())
print(df['Anxious Before Exams'].value_counts())
print(df['Academic Performance Change'].value_counts())
```

```
Gender
Male      475
Female    475
Other      50
Name: count, dtype: int64
Education Level
MTech      143
MSc        138
MA          129
Class 11    88
Class 9     87
Class 10    87
BSc         85
BTech       84
BA          62
```

```

Class 8      50
Class 12     47
Name: count, dtype: int64
Stress Level
Medium      492
Low         327
High        181
Name: count, dtype: int64
Anxious Before Exams
Yes         513
No          487
Name: count, dtype: int64
Academic Performance Change
Same        399
Improved    303
Declined    298
Name: count, dtype: int64

```

```

[9]: # Merging values in Education Level
def categorizeEducationLevel(edu_lvl):
    if edu_lvl in ['MTech', 'MSc', 'MA']:
        return "Masters"
    elif edu_lvl in ['BSc', 'BTech', 'BA']:
        return "Bachelors"
    elif edu_lvl in ['Class 8']:
        return "Middle School"
    else:
        return "High School"

```

```

[10]: df['Education Level'] = df['Education Level'].map(categorizeEducationLevel)
df.head()

```

```

[10]:
   Name  Gender  Age  Education Level  Screen Time (hrs/day) \
0  Aarav   Male   15   Middle School              7.1
1  Meera  Female   25         Masters              3.3
2  Ishaan  Male   20     Bachelors              9.5
3  Aditya  Male   20     Bachelors             10.8
4  Anika  Female   17   High School              2.8

   Sleep Duration (hrs)  Physical Activity (hrs/week)  Stress Level \
0                8.9                9.3         Medium
1                5.0                0.2         Medium
2                5.4                6.2         Medium
3                5.6                5.5           High
4                5.4                3.1         Medium

```

Anxious Before Exams Academic Performance Change

0	No	Same
1	No	Same
2	No	Same
3	Yes	Same
4	Yes	Same

```
[11]: print(df['Education Level'].value_counts())
```

```
Education Level
Masters      410
High School  309
Bachelors    231
Middle School  50
Name: count, dtype: int64
```

Replacing the values in columns with numerical values

- Gender: Male = 1, Female = 0, Other = 2
- Education Level: Masters = 0, Bachelors = 1, High School = 2, Middle School = 3
- Stress Level: Low = 0, medium = 1, high = 2
- Anxious before exam: Yes = 1, No = 0
- Academic Performance Change: Same = 0, Improved = 1, Declined = 2

```
[12]: # df['Gender'].replace({'Male':1, 'Female':0, 'Other':2}, inplace=True)
# df['Education Level'].replace({'Bachelors':1, 'Masters':0, 'High School':2,
#                               ↪ 'Middle School':3}, inplace=True)
# df['Stress Level'].replace({'Medium':1, 'Low':0, 'High':2}, inplace=True)
# df['Anxious Before Exams'].replace({'Yes':1, 'No':0}, inplace=True)
# df['Academic Performance Change'].replace({'Same':0, 'Improved':1, 'Declined':
#                               ↪ 2}, inplace=True)
```

```
[13]: df.head()
```

```
[13]:
```

	Name	Gender	Age	Education Level	Screen Time (hrs/day)	\
0	Aarav	Male	15	Middle School	7.1	
1	Meera	Female	25	Masters	3.3	
2	Ishaan	Male	20	Bachelors	9.5	
3	Aditya	Male	20	Bachelors	10.8	
4	Anika	Female	17	High School	2.8	

	Sleep Duration (hrs)	Physical Activity (hrs/week)	Stress Level	\
0	8.9	9.3	Medium	
1	5.0	0.2	Medium	
2	5.4	6.2	Medium	
3	5.6	5.5	High	
4	5.4	3.1	Medium	

Anxious Before Exams Academic Performance Change

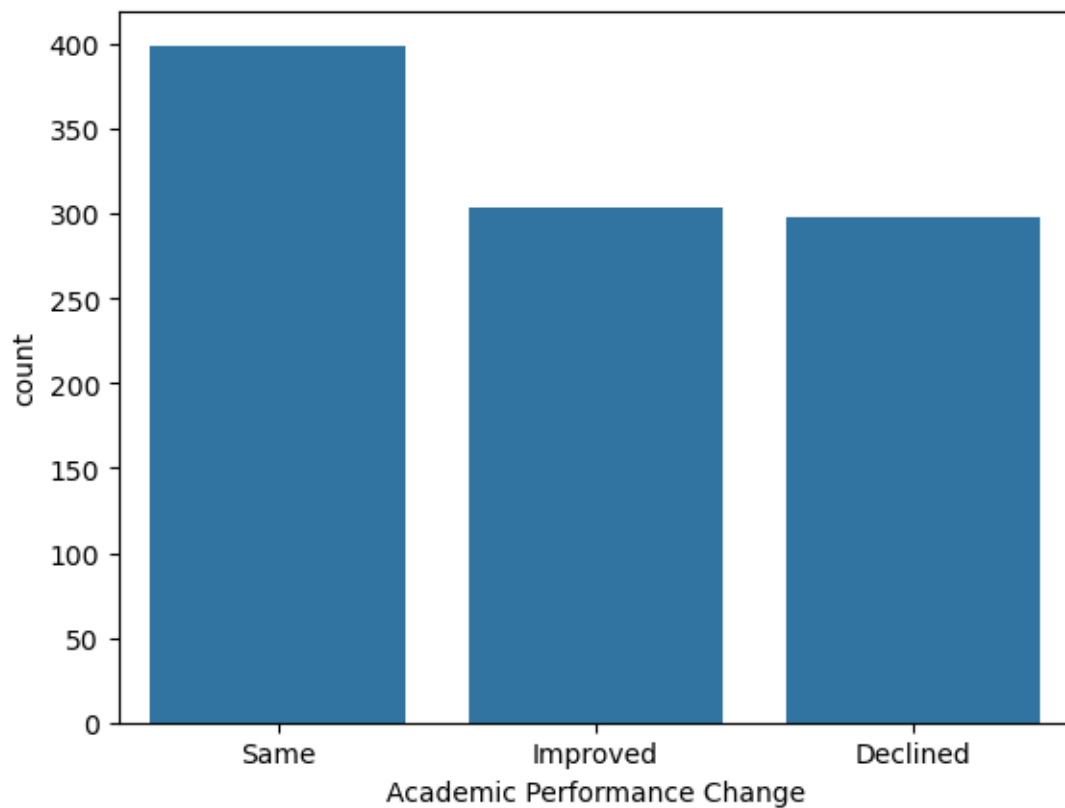
0	No	Same
1	No	Same
2	No	Same
3	Yes	Same
4	Yes	Same

1.2 2. Exploratory Data Analysis

In this section, I'll look at the relationship between the academic performance change (target variable) and the other variables. I will also look at the distribution of the variables across the dataset to understand the data in a better way

1.2.1 2.1 Analyzing Academic Performance Change

```
[14]: sns.countplot(df, x="Academic Performance Change")
plt.show()
```



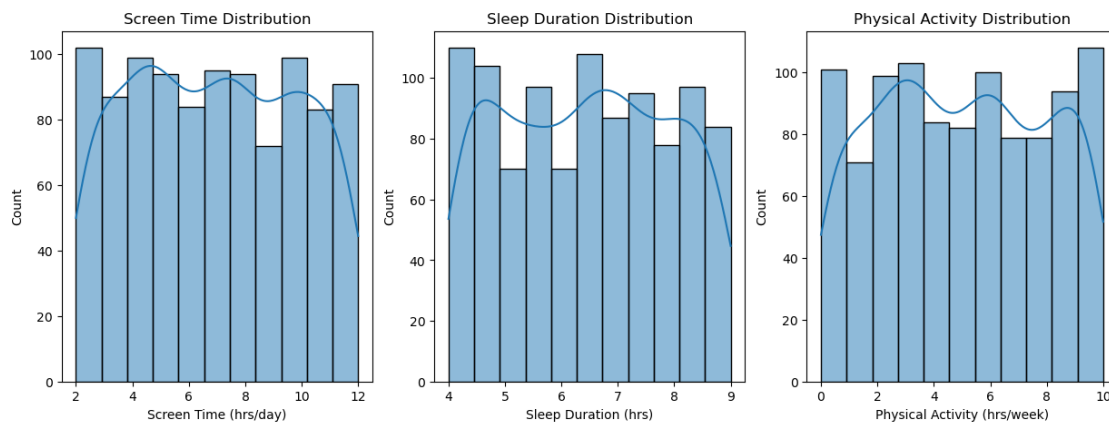
1.2.2 2.2 Numerical variables distributions

```
[15]: df.columns
```

```
[15]: Index(['Name', 'Gender', 'Age', 'Education Level', 'Screen Time (hrs/day)',  
          'Sleep Duration (hrs)', 'Physical Activity (hrs/week)', 'Stress Level',  
          'Anxious Before Exams', 'Academic Performance Change'],  
         dtype='object')
```

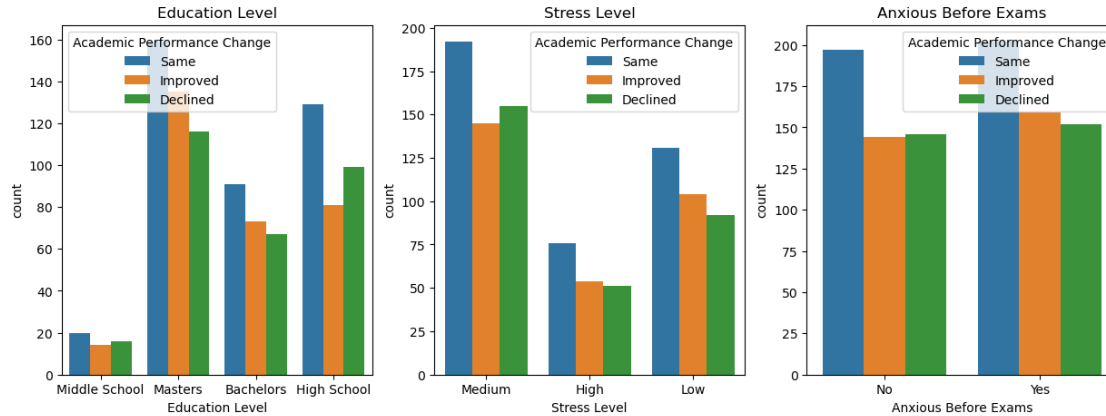
```
[16]: fig, ax = plt.subplots(1,3,figsize=(15,5))  
sns.histplot(df['Screen Time (hrs/day)'],ax=ax[0],kde=True).set_title('Screen_␣  
    ↪Time Distribution')  
sns.histplot(df['Sleep Duration (hrs)'],ax=ax[1],kde=True).set_title('Sleep_␣  
    ↪Duration Distribution')  
sns.histplot(df['Physical Activity (hrs/week)'],ax=ax[2],kde=True).  
    ↪set_title('Physical Activity Distribution')
```

```
[16]: Text(0.5, 1.0, 'Physical Activity Distribution')
```



```
[17]: fig, ax = plt.subplots(1,3,figsize=(15,5))  
sns.countplot(x = 'Education Level', data = df, ax=ax[0], hue='Academic_␣  
    ↪Performance Change').set_title('Education Level')  
sns.countplot(x = 'Stress Level', data = df, ax=ax[1], hue='Academic_␣  
    ↪Performance Change').set_title('Stress Level')  
sns.countplot(x = 'Anxious Before Exams', data = df, ax=ax[2], hue='Academic_␣  
    ↪Performance Change').set_title('Anxious Before Exams')
```

```
[17]: Text(0.5, 1.0, 'Anxious Before Exams')
```



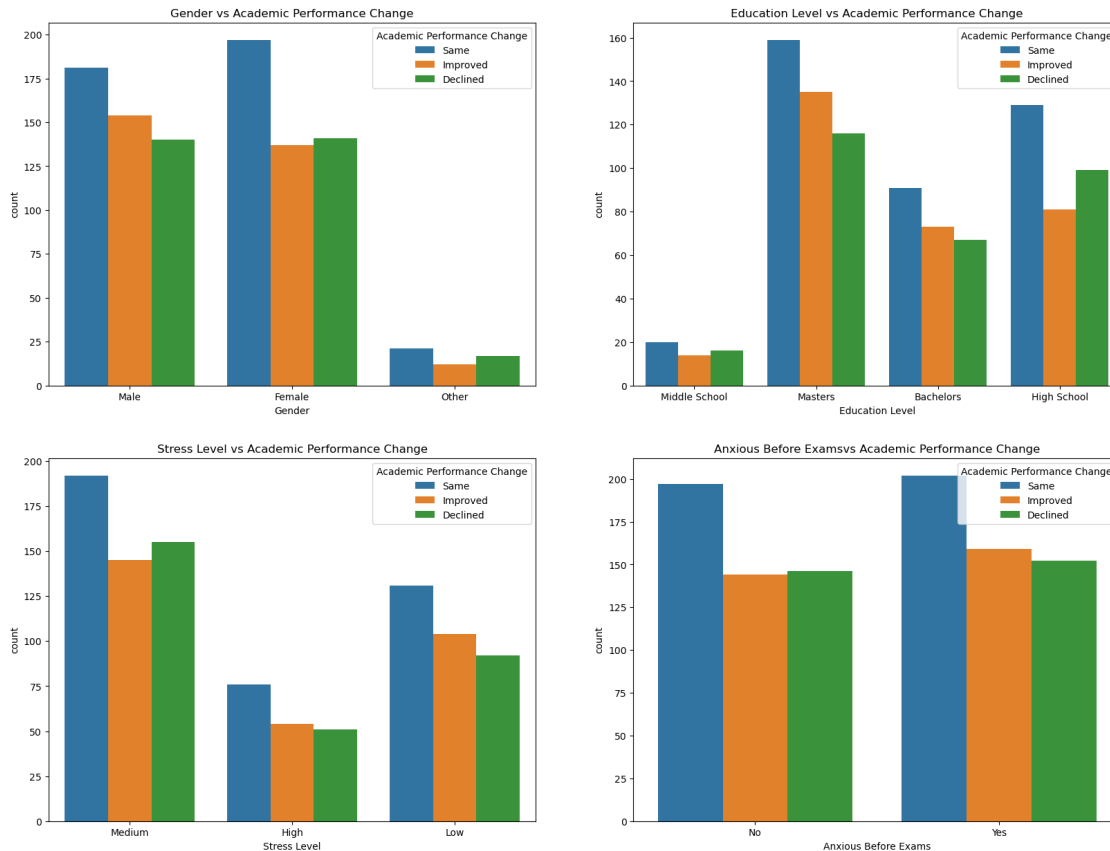
- 1st graph: it seems like no matter which education level a person has, his academic performance would majorly remain the same. One interesting observation here is that if a person only has middle or high school degree, his academic performance actually declines more than remaining the same. This could be due to a variety of reasons.
- 2nd graph:
- 3rd graph:

Unfortunately, we can't really figure out any salient relationships and patterns from graphs above. Let's dive in a little deeper on relationships between two variables

Categorical Variables vs. Academic Performance Change

```
[18]: fig, ax = plt.subplots(2,2,figsize=(20,15))
ax = ax.flatten()
sns.countplot(x = 'Gender', data = df, ax=ax[0], hue = 'Academic Performance_↵
↵Change').set_title('Gender vs Academic Performance Change')
sns.countplot(x = 'Education Level', data = df, ax=ax[1], hue = 'Academic_↵
↵Performance Change').set_title('Education Level vs Academic Performance_↵
↵Change')
sns.countplot(x = 'Stress Level', data = df, ax=ax[2], hue = 'Academic_↵
↵Performance Change').set_title('Stress Level vs Academic Performance Change')
sns.countplot(x = 'Anxious Before Exams', data = df, ax=ax[3], hue = 'Academic_↵
↵Performance Change').set_title('Anxious Before Examsvs Academic Performance_↵
↵Change')
```

```
[18]: Text(0.5, 1.0, 'Anxious Before Examsvs Academic Performance Change')
```

- 1st graph: by graphing relationship between gender vs academic performance change, we can see there is no apparent influence from gender on academic performance change
- 2nd graph: similar to previous plot, Masters degree students have the highest count of academic performance improvement
- 3rd graph: we can tell that if a student is kind of stressed before exam (medium), then he will do well academically. This makes logical sense in a real-world context, as if student doesn't care about school, he tends to have low stress level and therefore shows low improvement academically. In contrast, if student stresses about school too much, he would also do poorly as he is too stressed.
- 4th graph: no apparent relationship

1.2.3 2.3 Data Preprocessing

Label Encoding the Categorical Variables

```
[19]: le = LabelEncoder()
cols = ['Gender', 'Education Level', 'Stress Level', 'Anxious Before_
↪Exams', 'Academic Performance Change']
for i in cols:
    le.fit(df[i])
    df[i]=le.transform(df[i])
```

```
print(i,df[i].unique())
```

```
Gender [1 0 2]
Education Level [3 2 0 1]
Stress Level [2 0 1]
Anxious Before Exams [0 1]
Academic Performance Change [2 1 0]
```

Correlation Matrix Heatmap

```
[20]: df.columns
```

```
[20]: Index(['Name', 'Gender', 'Age', 'Education Level', 'Screen Time (hrs/day)',
          'Sleep Duration (hrs)', 'Physical Activity (hrs/week)', 'Stress Level',
          'Anxious Before Exams', 'Academic Performance Change'],
          dtype='object')
```

```
[21]: df.head()
```

```
[21]:
```

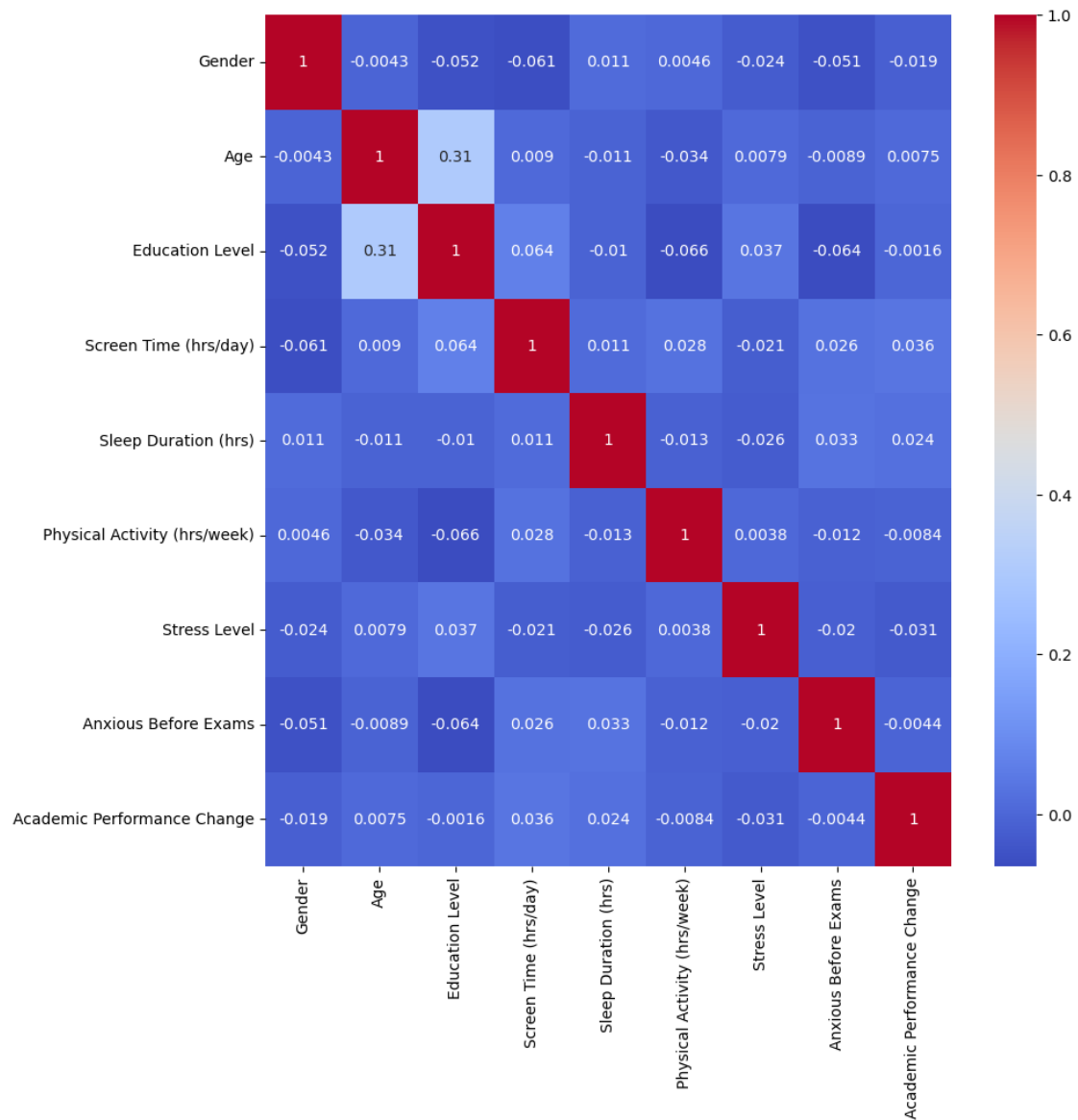
	Name	Gender	Age	Education Level	Screen Time (hrs/day)	\
0	Aarav	1	15	3	7.1	
1	Meera	0	25	2	3.3	
2	Ishaan	1	20	0	9.5	
3	Aditya	1	20	0	10.8	
4	Anika	0	17	1	2.8	

	Sleep Duration (hrs)	Physical Activity (hrs/week)	Stress Level	\
0	8.9	9.3	2	
1	5.0	0.2	2	
2	5.4	6.2	2	
3	5.6	5.5	0	
4	5.4	3.1	2	

	Anxious Before Exams	Academic Performance Change
0	0	2
1	0	2
2	0	2
3	1	2
4	1	2

```
[22]: plt.figure(figsize=(10,10))
      sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
```

```
[22]: <Axes: >
```



```
[23]: df.corr(numeric_only=True)['Academic Performance Change'].sort_values()
```

```
[23]: Stress Level          -0.030854
      Gender              -0.018660
      Physical Activity (hrs/week) -0.008395
      Anxious Before Exams    -0.004377
      Education Level        -0.001629
      Age                   0.007487
      Sleep Duration (hrs)    0.023520
      Screen Time (hrs/day)   0.036491
      Academic Performance Change 1.000000
```

Name: Academic Performance Change, dtype: float64

1.3 3. Modeling Training

1.3.1 3.1 Split Data into training and testing set

```
[24]: X_train, X_test, y_train, y_test = train_test_split(df.drop('Academic_
↳Performance Change', axis=1), df['Academic Performance Change'], test_size=0.
↳2, random_state=0)
```

```
[25]: X_train.shape, X_test.shape
```

```
[25]: ((800, 9), (200, 9))
```

```
[26]: print(X_train.dtypes)
```

```
Name                object
Gender              int64
Age                int64
Education Level     int64
Screen Time (hrs/day) float64
Sleep Duration (hrs) float64
Physical Activity (hrs/week) float64
Stress Level        int64
Anxious Before Exams int64
dtype: object
```

```
[27]: X_train = X_train.drop(columns=['Name'])
X_test = X_test.drop(columns=['Name'])
print(X_train.shape, X_test.shape)
X_train.head()
```

```
(800, 8) (200, 8)
```

```
[27]:
```

	Gender	Age	Education Level	Screen Time (hrs/day)	\
687	0	24	2	11.0	
500	1	23	2	9.7	
332	1	20	0	2.9	
979	1	19	0	9.9	
817	1	23	2	4.7	

	Sleep Duration (hrs)	Physical Activity (hrs/week)	Stress Level	\
687	7.4	9.4	0	
500	7.8	3.2	1	
332	8.1	5.4	1	
979	4.2	5.4	2	
817	7.3	5.3	0	

	Anxious Before Exams
687	0
500	0
332	1
979	1
817	0

1.3.2 3.2 Model Building

I will be using the following models to predict the academic performance change for students: * Random Forest Classifier * Decision Tree Classifier * Logistic Regression * K Nearest Neighbors

Random Forest Classifier

```
[28]: rfc = RandomForestClassifier()
```

```
[29]: #Parameter grid
param_grid = {
    'max_depth': [4,8,12,16],
    'min_samples_leaf': [2,4,6,8],
    'min_samples_split': [2,4,6,8],
    'criterion': ['gini', 'entropy'],
    'random_state': [0,42]
}

#GridSearchCV object
grid = GridSearchCV(estimator=rfc, param_grid=param_grid, cv=5, n_jobs=-1,
    verbose=0, scoring='accuracy')

# Convert categoricals
X_train = pd.get_dummies(X_train, drop_first=True)
X_test = pd.get_dummies(X_test, drop_first=True)
X_train, X_test = X_train.align(X_test, join='left', axis=1, fill_value=0)

#Fitting the model
grid.fit(X_train, y_train)

#Best parameters
print('Best parameters: ', grid.best_params_)
```

```
Best parameters: {'criterion': 'entropy', 'max_depth': 4, 'min_samples_leaf': 8, 'min_samples_split': 2, 'random_state': 0}
```

```
/opt/anaconda3/lib/python3.12/site-packages/numpy/ma/core.py:2820:
```

```
RuntimeWarning: invalid value encountered in cast
    _data = np.array(data, dtype=dtype, copy=copy,
```

```
[30]: #Random Forest Classifier Object
rfc = RandomForestClassifier(criterion='gini', max_depth=4, min_samples_leaf=8,
    ↪min_samples_split=2, random_state=0)

#Fitting the model
rfc.fit(X_train, y_train)
```

```
[30]: RandomForestClassifier(max_depth=4, min_samples_leaf=8, random_state=0)
```

```
[31]: #Training accuracy
print('Training accuracy: ', rfc.score(X_train, y_train))
```

Training accuracy: 0.46875

```
[32]: #predicting the test set results
rfc_pred = rfc.predict(X_test)
```

Decision Tree Classifier

```
[33]: #Decision Tree Classifier Object
dtc = DecisionTreeClassifier()
```

```
[34]: param_grid = {
    'max_depth': [2,4,6,8],
    'min_samples_leaf': [2,4,6,8],
    'min_samples_split': [2,4,6,8],
    'criterion': ['gini', 'entropy'],
    'random_state': [0,42]
}

#GridSearchCV object
grid = GridSearchCV(estimator=dtc, param_grid=param_grid, cv=5, n_jobs=-1,
    ↪verbose=0, scoring='accuracy')

#Fitting the model
grid.fit(X_train, y_train)

#Best parameters
print('Best parameters: ', grid.best_params_)
```

Best parameters: {'criterion': 'entropy', 'max_depth': 4, 'min_samples_leaf': 2, 'min_samples_split': 2, 'random_state': 0}

/opt/anaconda3/lib/python3.12/site-packages/numpy/ma/core.py:2820:

RuntimeWarning: invalid value encountered in cast
 _data = np.array(data, dtype=dtype, copy=copy,

```
[35]: #Decision Tree Classifier Object
dtc = DecisionTreeClassifier(criterion='gini', max_depth=4, min_samples_leaf=2,
    ↪ min_samples_split=2, random_state=0, class_weight='balanced')

#Fitting the model
dtc.fit(X_train, y_train)
```

```
[35]: DecisionTreeClassifier(class_weight='balanced', max_depth=4, min_samples_leaf=2,
    random_state=0)
```

```
[36]: #Training accuracy
print('Training accuracy: ', dtc.score(X_train, y_train))
```

Training accuracy: 0.42375

```
[37]: #predicting the test set results
dtc_pred = dtc.predict(X_test)
```

Logistic Regression

```
[38]: #Logistic Regression Object
lr = LogisticRegression()
```

```
[39]: #fitting the model
lr.fit(X_train, y_train)
```

/opt/anaconda3/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:469: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
[39]: LogisticRegression()
```

```
[40]: #Training accuracy
lr.score(X_train, y_train)
```

```
[40]: 0.40125
```

```
[41]: #predicting the test set results
lr_pred = lr.predict(X_test)
```

K Nearest Neighbors

```
[43]: #KNN Classifier Object
knn = KNeighborsClassifier()
```

```
[44]: #fitting the model
knn.fit(X_train, y_train)
```

```
[44]: KNeighborsClassifier()
```

```
[45]: #training accuracy
knn.score(X_train, y_train)
```

```
[45]: 0.535
```

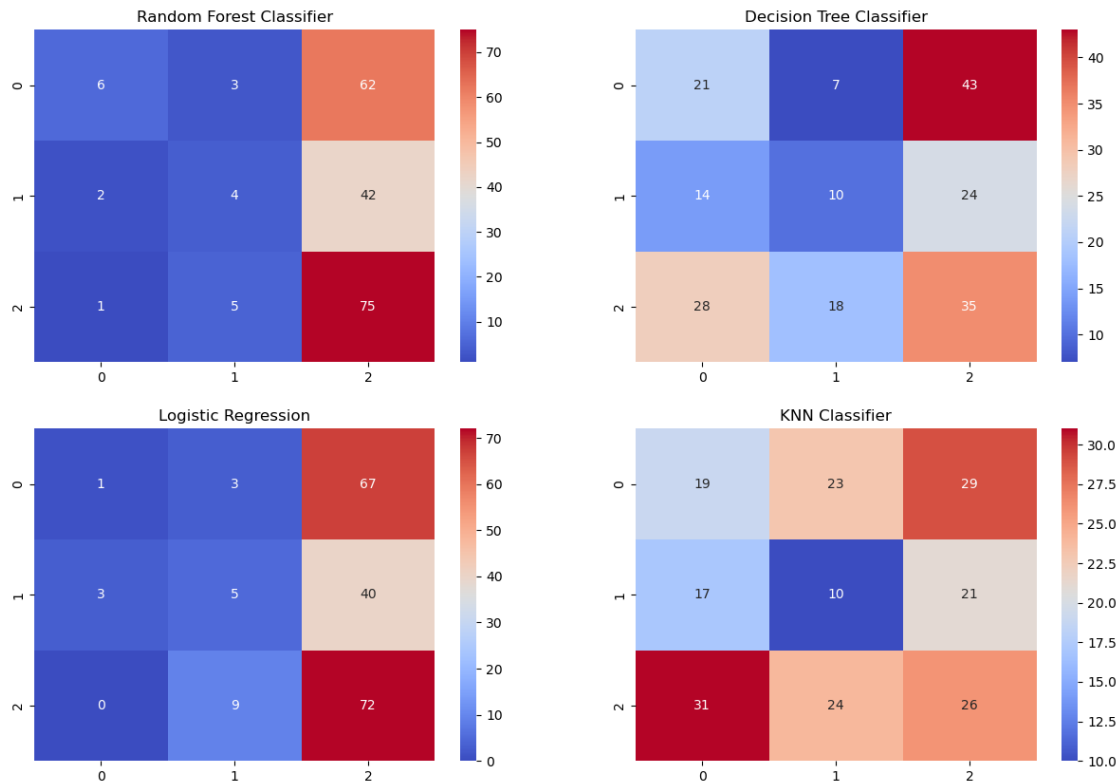
```
[46]: #predicting the test set results
knn_pred = knn.predict(X_test)
```

1.4 4. Model Evaluation

```
[47]: fig, ax = plt.subplots(2,2,figsize=(15,10))
sns.heatmap(confusion_matrix(y_test, rfc_pred), annot=True, cmap='coolwarm',
            ↪ax=ax[0,0]).set_title('Random Forest Classifier')
sns.heatmap(confusion_matrix(y_test, dtc_pred), annot=True, cmap='coolwarm',
            ↪ax=ax[0,1]).set_title('Decision Tree Classifier')
sns.heatmap(confusion_matrix(y_test, lr_pred), annot=True, cmap='coolwarm',
            ↪ax=ax[1,0]).set_title('Logistic Regression')
sns.heatmap(confusion_matrix(y_test, knn_pred), annot=True, cmap='coolwarm',
            ↪ax=ax[1,1]).set_title('KNN Classifier')
```

The history saving thread hit an unexpected error (OperationalError('attempt to write a readonly database')).History will not be written to the database.

```
[47]: Text(0.5, 1.0, 'KNN Classifier')
```

```
[48]: #classification report
print('Random Forest Classifier: \n', classification_report(y_test, rfc_pred))
print('Decision Tree Classifier: \n', classification_report(y_test, dtc_pred))
print('Logistic Regression: \n', classification_report(y_test, lr_pred))
print('KNN Classifier: \n', classification_report(y_test, knn_pred))
```

Random Forest Classifier:

	precision	recall	f1-score	support
0	0.67	0.08	0.15	71
1	0.33	0.08	0.13	48
2	0.42	0.93	0.58	81
accuracy			0.42	200
macro avg	0.47	0.36	0.29	200
weighted avg	0.49	0.42	0.32	200

Decision Tree Classifier:

	precision	recall	f1-score	support
0	0.33	0.30	0.31	71
1	0.29	0.21	0.24	48

2	0.34	0.43	0.38	81
accuracy			0.33	200
macro avg	0.32	0.31	0.31	200
weighted avg	0.33	0.33	0.32	200

Logistic Regression:

	precision	recall	f1-score	support
0	0.25	0.01	0.03	71
1	0.29	0.10	0.15	48
2	0.40	0.89	0.55	81
accuracy			0.39	200
macro avg	0.32	0.34	0.24	200
weighted avg	0.32	0.39	0.27	200

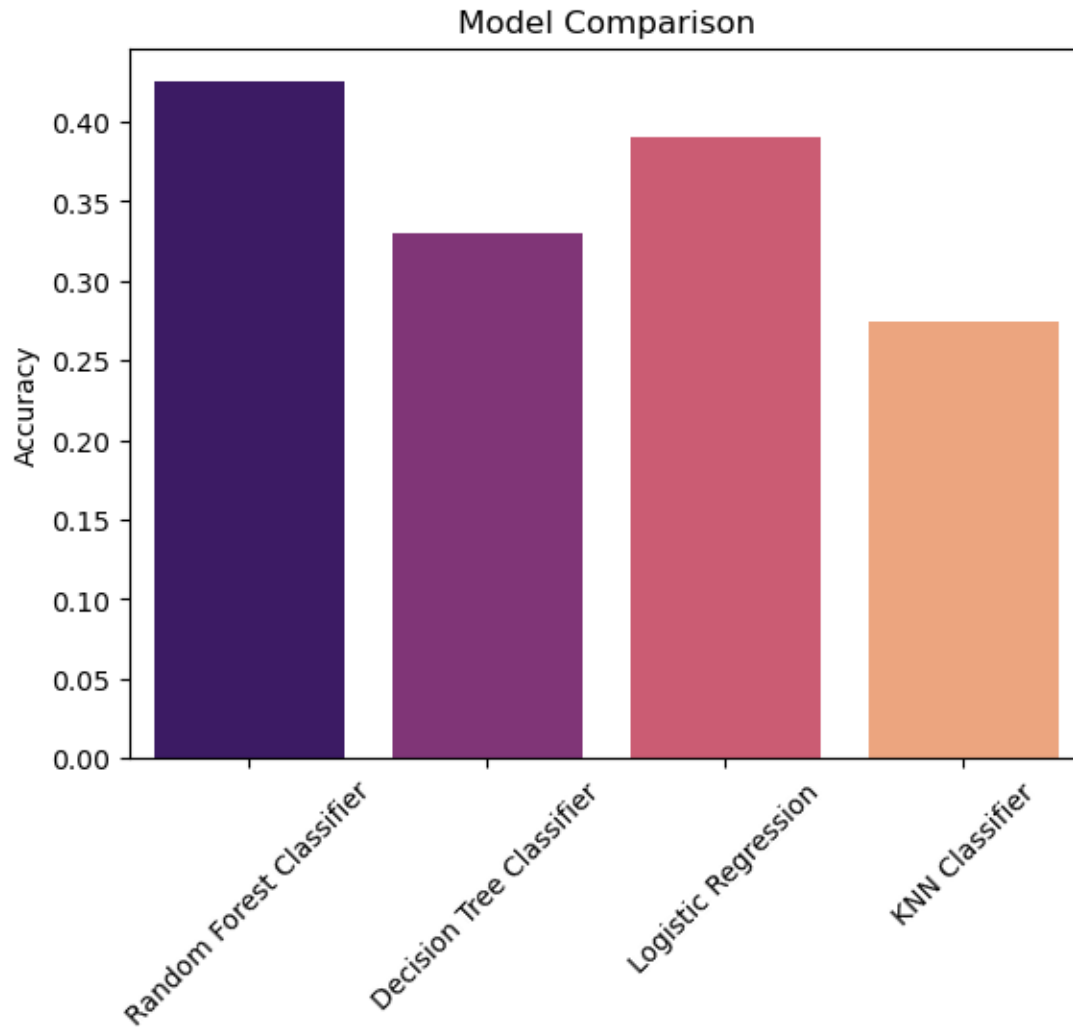
KNN Classifier:

	precision	recall	f1-score	support
0	0.28	0.27	0.28	71
1	0.18	0.21	0.19	48
2	0.34	0.32	0.33	81
accuracy			0.28	200
macro avg	0.27	0.27	0.27	200
weighted avg	0.28	0.28	0.28	200

1.4.1 Model Comparison

```
[52]: models = ['Random Forest Classifier', 'Decision Tree Classifier', 'Logistic_
↳Regression', 'KNN Classifier']
accuracy = [accuracy_score(y_test, rfc_pred), accuracy_score(y_test, dtc_pred),
↳accuracy_score(y_test, lr_pred), accuracy_score(y_test, knn_pred)]
sns.barplot(x=models, y=accuracy, hue=models, palette='magma', legend=False).
↳set_title('Model Comparison')
plt.xticks(rotation=45)
plt.ylabel('Accuracy')
```

```
[52]: Text(0, 0.5, 'Accuracy')
```



1.5 5. Conclusion

In this project, our primary goal is to analyze a dataset which contains responses from students regarding their mental health status during the era of online learning. We want to find out what factors affect students' academic performances with machine learning techniques.

By doing exploratory data analysis, I found that the academic performance is most closely related with screen time, as their correlation value is the highest. Then I started to train models with four distinct ML techniques. At the end, by comparing their performances, it turns out that Random Forest model obtains the highest accuracy of nearly 50%, while the other three are closely trailing in their accuracies.